PYTHON 学习档案

第 11-14 天学习总结

对象协议 → 并发工具 → asyncio → 订单同步实战

覆盖范围	记录日期	文件状态
第 11-14 天	2026-09-04 至 2026-09-09	9 个源文件；4 份作业已修正并验收

整理原则：只依据实际读取到的 Python 文件；讲义中的背景说明与修正后的作业答案分开标识。

目录

目录 2

文件扫描与归属说明 4

修正前发现的问题 . . . 4

第 11 天：对象协议、魔术方法、MRO 与 Mixin 5

学习主题 . . . 5

核心知识点 . . . 5

代码逻辑解析 . . . 5

易错点与注意事项 . . . 6

第 12 天：线程、进程、GIL 与并发工具 6

学习主题 . . . 6

核心知识点 . . . 6

代码逻辑解析 . . . 6

易错点与注意事项 . . . 7

第 13 天：asyncio、协程与结构化并发 7

学习主题 . . . 7

核心知识点 . . . 7

代码逻辑解析 . . . 8

易错点与注意事项 . . . 8

第 14 天：订单同步与批处理综合实战 8

学习主题 . . . 8

核心知识点 . . . 8

代码逻辑解析 . . . 9

易错点与注意事项 . . . 9

跨天综合复习 10

知识点总览 . . . 10

知识之间的联系 . . . 10

重点代码模式 . . . 10

常见错误汇总 . . . 10

复习清单 11

综合练习与参考答案 **11**

练习 1（基础） 11

练习 2（线程池） 11

练习 3（asyncio） 11

练习 4（数据与幂等） 12

练习 5（综合设计） 12

验收结果与学习文件状态 **12**

文件扫描与归属说明

扫描当前工作目录及子目录后，以目录日期、文件名中的 day
编号和文件首部主题三项交叉判断。四天文件均有明确编号，无需仅凭内容猜测。

天数 / 日期	实际扫描文件	文件名与首行标题
第 11 天 20260904	day_11_homework.py day_11_mixin.py day_11_object_oriented_magic_methods_mro_object_protocol.py	文件名均标 Day 11；讲义主题为魔术方法、MRO、Mixin；作业已重写并通过验收。
第 12 天 20260907	day_12_homework.py day_12_thread_process_GIL_concurrent_tools.py	文件名与首行均标 Day 12；主题为线程、进程、GIL 与并发工具；原作业仅有题目，现已补答。
第 13 天 20260908	day_13_asyncio_aync_await_async-concurrency.py day_13_homework.py	文件名标 Day 13；讲义内容为 asyncio、async/await 与异步并发；原作业仅有题目，现已补答。
第 14 天 20260909	day_14_homework.py day_14_order_sync_batch_processing_comprehensive_combat.py	文件名与主题均标 Day 14；综合订单同步、清洗、幂等与批处理；原作业仅有题目，现已补答。

文件状态说明
5 个讲义/示例文件的内容全部以注释形式保存，本身没有可执行入口；这不是语法错误。四份作业文件现为可运行实现，并保留了题目。

修正前发现的问题

- 第 11 天存在重复实现；早期版本把 __hash__ 写成接收 other 的比较函数，既不符合 __hash__(self) 协议，也混淆了哈希与相等。
- 旧 OrderKey 多了可变的 productList，字段也把 order_no 写成 order_id，无法满足不可变可哈希业务键的题目要求。
- 旧 OrderService 被声明成 Protocol，只有接口外形而没有组合依赖后的业务实现。
- 旧 Money 没有 frozen/slots、没有 ROUND_HALF_UP 两位小数归一化，且加法返回 Decimal 而非 Money。
- 第 12-14 天作业原文件只有题目与验收要求，没有答案代码。

第 11 天：对象协议、魔术方法、MRO 与 Mixin

学习主题

让领域对象通过特殊方法自然融入 Python 语法；理解 dataclass 与可哈希值对象；掌握 super() 沿 MRO 继续查找的真实含义；区分普通继承、Mixin 和组合。

核心知识点

- 语言协议映射：len → __len__，in → __contains__，for → __iter__，[] → __getitem__，+ → __add__。
- __repr__ 面向开发者与调试；__str__ 面向用户表达；语义清晰时才重载运算符。
- a == b 时必须保证 hash(a) == hash(b)；参与 set/dict key 的对象通常应保持不可变。
- dataclass 默认生成 __init__、__repr__、__eq__；frozen、slots、order 会进一步改变对象协议。
- super() 不是固定调用直接父类，而是在当前类之后沿 MRO 查找；菱形继承中 D(B,C) 的顺序为 D→B→C→A→object。
- 普通继承表达 is-a，组合表达 has-a/依赖，Mixin 提供小而单一的横向能力；Protocol 描述能力要求而不负责实现。

> **当天重点**
> Pythonic 不是“魔术方法越多越好”，而是对象行为符合语言直觉；复杂业务依赖优先组合。

代码逻辑解析

Order 把内部 items 列表已有的能力转交给语言协议：长度直接委托给 len(items)，迭代直接返回 iter(items)，成员检查先统一 SKU
的空格与大小写，索引/切片则原样委托给列表。这样既复用成熟行为，又让业务对象保留清晰语义。

```
class Order:
    def __len__(self) -> int:
        return len(self.items)

    def __iter__(self) -> Iterator[OrderItem]:
        return iter(self.items)

    def __contains__(self, sku: object) -> bool:
        if not isinstance(sku, str):
            return False
        target = sku.strip().upper()
        return any(
            item.sku.strip().upper() == target
            for item in self.items
        )

    def __getitem__(self, index: int | slice):
        return self.items[index]
```

Money 使用 frozen=True 保持值语义，__post_init__ 通过 object.__setattr__
完成一次合法的归一化；加法返回新的 Money，而不是泄露底层 Decimal。

```
@total_ordering
@dataclass(frozen=True, slots=True)
class Money:
    amount: Decimal

    def __post_init__(self) -> None:
        value = Decimal(str(self.amount)).quantize(
            Decimal("0.01"), rounding=ROUND_HALF_UP
        )
        object.__setattr__(self, "amount", value)

    def __add__(self, other: object) -> "Money":
        if not isinstance(other, Money):
            return NotImplemented
        return Money(self.amount + other.amount)
```

易错点与注意事项

- __hash__ 只接收 self 并返回整数；它不是两个对象的比较函数。
- __eq__ 与 __lt__ 面对不支持的类型应返回 NotImplemented，而不是假定对方一定同类。
- 切片返回 list，整数索引返回单个元素；overload 可让静态类型提示更准确。
- Mixin 方法重名由 MRO 决定，Mixin 应小、职责单一并避免携带复杂业务状态。
- 组合中的 Repository、Cache、Notifier 是完整依赖，不应伪装成多继承的“能力类”。

> **当天知识总结**
> 掌握“实现协议获得语言能力”；用不可变值对象维护金额和业务键；能解释 MRO，并在继承、Mixin、组合之间按语义做选择。

第 12 天：线程、进程、GIL 与并发工具

学习主题

先按瓶颈选择并发模型：I/O 密集任务使用线程池或异步，纯 Python CPU 密集任务通常使用进程池；围绕 Future、异常隔离、锁、超时和任务生命周期编写可靠并发代码。

核心知识点

- 并发表示多个任务在同一时间段推进；并行表示真正同时占用不同 CPU 核心。
- 常规 CPython 的 GIL 限制一个进程内同时执行 Python 字节码的线程数量，但 I/O 等待时线程仍然有价值。
- ThreadPoolExecutor 适合批量同步 I/O；submit 返回 Future，map 按输入顺序产出，as_completed 按完成顺序产出。
- Future 中的异常通常在 result() 时重新抛出；必须逐任务处理，才能做到失败隔离。
- Lock 保护同一进程内线程共享状态的临界区；RLock 允许同一线程重入；Event 用于明确传递停止/就绪信号。
- ProcessPoolExecutor 可利用多核，但进程独立、传参需 pickle；任务函数应放模块顶层并使用 __main__ 入口保护。

代码逻辑解析

正确的批量模式是先提交全部任务，用 Future→site_id 映射保留业务上下文，然后按完成顺序分别取结果。异常只登记到失败站点，成功站点继续计入总数。

```
with ThreadPoolExecutor(max_workers=3) as executor:
    future_to_site = {
        executor.submit(fetch_order_count, site_id,
                        fail_site_3=True): site_id
        for site_id in [1, 2, 3, 4, 5]
    }
    for future in as_completed(future_to_site):
        site_id = future_to_site[future]
        try:
            total_orders += future.result()
        except Exception as exc:
            failed_sites[site_id] = str(exc)
```

共享计数器的 counter += 1
是业务上的读-改-写；不能把某个解释器版本中“碰巧没有出错”当作线程安全保证。进程池则把四个 CPU 任务交给独立进程，入口保护避免 Windows 子进程重复创建进程池。

```
def calculate_sum(limit: int) -> int:
    return sum(i * i for i in range(limit))

def main() -> None:
    with ProcessPoolExecutor(max_workers=4) as executor:
        results = list(executor.map(
            calculate_sum, [100_000] * 4
        ))

if __name__ == "__main__":
    main()
```

易错点与注意事项

- 不要在每次 submit 后立刻 result()，否则线程池会退化成近似串行。
- GIL 既不等于线程无用，也不等于共享业务数据无需 Lock。
- Future.result(timeout) 只限制等待时间，不会强制杀死已运行任务；cancel 也通常只能取消未开始任务。
- 线程池大小受 API 限流、连接池、数据库容量、CPU/内存和压测结果共同约束，不是越大越好。
- 局部函数和 lambda 通常不适合作为进程池任务，因为标准进程池需要序列化函数及参数。

> **当天知识总结**
> 会判断 I/O 与 CPU 瓶颈；会用线程池 + as_completed 做业务级失败隔离；会用 Lock 保护共享状态；会安全启动进程池。

第 13 天：asyncio、协程与结构化并发

学习主题

理解协程由事件循环协作式调度，掌握 Task、gather、TaskGroup、超时、取消、Semaphore、Queue，以及用 to_thread 接入同步阻塞 I/O。

核心知识点

- async def 定义协程函数；调用后得到协程对象；只有 await 或交给 Task 后才会执行。
- await 是等待并在必要时让出执行权，不等于自动创建并发；连续 await 仍然通常是顺序执行。
- create_task 把协程交给事件循环调度；gather 适合收集结果且结果顺序与传入顺序一致。
- TaskGroup 管理一组共同生命周期的任务；子任务未处理异常会触发其他任务取消和统一收尾。
- asyncio.timeout/wait_for 提供超时；取消通常在等待点抛出 CancelledError，清理后应继续传播。

- Semaphore 限制进入资源区的并发数；Queue(maxsize) 提供背压；Lock、Event、Queue 分别处理互斥、通知和生产消费。

代码逻辑解析

顺序版本逐个 await 五个 1 秒任务，等待时间累加为约 5 秒；gather
同时调度五个协程，等待区间重叠，总耗时接近最长的单个任务，即约 1 秒。

```
sequential = [
    await numbered_work(n, 1.0)
    for n in range(1, 6)
]

concurrent = await asyncio.gather(*(
    numbered_work(n, 1.0)
    for n in range(1, 6)
))
```

失败隔离练习给每个请求单独套 wait_for，并用 gather(return_exceptions=True) 收集正常值、ValueError 和 TimeoutError；Semaphore 的 finally 保证活跃计数总能回退。

```
async def guarded_request(request_id: int) -> int:
    return await asyncio.wait_for(
        simulated_request(request_id), timeout=1.0
    )

outcomes = await asyncio.gather(
    *(guarded_request(i) for i in range(1, 11)),
    return_exceptions=True,
)
```

生产者消费者中，Queue(maxsize=5) 让生产者在队列满时等待；三个消费者各需要一个 None 哨兵；每次 get 都必须在 finally 中调用 task_done，queue.join 才能正确结束。

易错点与注意事项

- 只创建协程对象却不 await，会出现 coroutine was never awaited 警告。
- 在 async def 中调用 time.sleep、requests 或同步数据库查询仍会阻塞事件循环。
- Semaphore 限制的是临界资源区并发，不限制已创建 Task 的总数；海量任务还需有界队列。
- 不要吞掉 CancelledError；资源释放应放在 finally 或 async with。
- to_thread 适合桥接同步阻塞 I/O，不是纯 Python CPU 密集计算的通用加速器。

> **当天知识总结**
> 能区分协程、Task 与
> Future；能主动组织并发，设置超时与失败边界，限制下游并发，并用有界队列组织可退出的生产者消费者。

第 14 天：订单同步与批处理综合实战

学习主题

把前 13 天知识组合成可测试的订单同步流程：外部接口拉取 → 数据清洗与校验 → 本轮去重 → 分批保存 → 幂等控制 → 统计结果；重点关注数据正确性、并发边界、失败处理与可测试性。

核心知识点

- 先在系统边界把不可信 dict 转换成不可变 Order；金额从字符串构造 Decimal 并用 ROUND_HALF_UP 保留两位。

- 分离 OrderApi、clean_order、OrderRepository、OrderSyncService；异步是执行方式，分层是代码结构。
- Semaphore 控制同时请求店铺数，wait_for 控制单次接口超时，预期接口错误按店铺隔离。
- 重试只包裹可恢复的 HTTP 超时；订单校验失败属于数据问题，不应重试接口。
- 本轮 seen 解决去重；Repository 的业务唯一键解决重复执行幂等；真实生产仍需数据库唯一索引/UPSERT。
- batched 逐批保存并记录批次大小；SyncResult 区分
  fetched、valid、invalid、duplicates、saved、failed_sites、retries。

代码逻辑解析

清洗阶段先完成类型转换和金额量化，再执行明确的业务校验。quantity 排除 bool
和非整数浮点值，只接受正整数或整数字符串；total_amount 由归一化后的 amount 乘数量得到。

```
@dataclass(frozen=True, slots=True)
class Order:
    site_id: int
    order_id: str
    sku: str
    amount: Decimal
    quantity: int

    @property
    def total_amount(self) -> Decimal:
        return self.amount * self.quantity

amount = Decimal(str(raw["amount"])).quantize(
    Decimal("0.01"), rounding=ROUND_HALF_UP
)
```

每个店铺在 Semaphore 范围内执行接口调用；最多允许 3 次额外重试。只有 TimeoutError
进入指数退避，OrderApiError 直接隔离，清洗错误发生在拉取结束后，因此不会触发接口重试。

```
for attempt in range(self.max_retries + 1):
    try:
        orders = await asyncio.wait_for(
            self.api.fetch_orders(site_id),
            timeout=self.timeout,
        )
        return site_id, orders, retries_used
    except TimeoutError:
        if attempt == self.max_retries:
            return site_id, None, retries_used
        retries_used += 1
        await asyncio.sleep(base_delay * (2 ** attempt))
```

同一服务连续同步两次时，两次都能得到 5 条通过校验且本轮去重后的 valid 订单；第一次 Repository 新增 5 条，第二次业务键已存在，所以 saved 为 0。这就是 valid 与 saved 可能不同的原因。

易错点与注意事项

- Decimal('NaN') 可构造成功，仍需 is_finite()；不要先把金额转成 float。
- 去重只保证本批次不重复，幂等保证同一同步动作重复执行不产生额外持久化结果。
- 重试整个“拉取-保存”流程可能在保存已成功后重复写入；重试边界必须足够小。
- except Exception: pass 会吞掉程序错误；业务错误、外部接口错误和程序缺陷应分层处理。
- 一次性 gather 所有店铺并保留全部订单会放大内存；大规模场景需分页、有界队列与固定 worker。

> **当天知识总结**
> 能够解释各模块职责和完整数据流；能用异步并发安全拉取、按规则清洗、去重、分批并实现幂等；能用测试证明重试、并发上限和重复执行行为。

跨天综合复习

知识点总览

主题	第 11-14 天的关键能力
对象与类型	特殊方法、dataclass、frozen/slots、Protocol、Decimal 值对象
并发模型	I/O 与 CPU 瓶颈判断；线程池、asyncio、进程池的选择
任务与结果	Future、Task、gather、TaskGroup、as_completed、异常回收
并发安全	threading.Lock、asyncio.Lock、Semaphore、Event、Queue/背压
可靠性	超时、取消、重试边界、失败隔离、finally 清理
数据工程	边界清洗、业务校验、去重、幂等、批处理、统计与测试
设计原则	语义自然的对象协议；组合优于继承；分层与依赖注入

知识之间的联系

第 11 天先让 Order、Money、OrderKey 具有可靠的对象语义；第 12 天学习如何并发执行同步 I/O 和 CPU 任务；第 13 天把大量可异步化 I/O 交给事件循环，并加入限流、超时和队列；第 14 天把模型、并发和可靠性组合成完整订单同步服务。

```
untrusted dict
    -> clean_order / validation
    -> immutable Order
    -> concurrent fetch boundary
    -> in-batch deduplication
    -> batched persistence
    -> idempotent repository
    -> SyncResult metrics
```

重点代码模式

模式	适用场景	关键写法
对象协议委托	领域对象包装集合	__len__/__iter__/__contains__/__getitem__ 委托内部 list
Future 业务映射	同步 SDK 的多站点 I/O	future_to_site + as_completed + 单任务 try/except
异步限流	异步 HTTP/数据库资源	async with Semaphore: await I/O
失败收集	每个任务允许独立成功/失败	gather(..., return_exceptions=True) 或任务内部转换状态
有界生产消费	数量很大，不能一次性创建/保留	Queue(maxsize) + 固定 worker + sentinel + task_done/join
幂等批量写入	同步任务可能重跑	业务唯一键 + 批次 + 数据库唯一约束/UPSERT

常见错误汇总

- 协议错误：__hash__ 写成比较函数、加法返回底层数值而非同类值对象、in 未处理非字符串输入。
- 并发退化：提交一个 Future 就立即等待；连续 await 却误以为已经并发。
- 错误安全感：依赖 GIL 代替锁；把 async def 内的同步阻塞调用误认为异步。
- 资源失控：线程/Task 数量无限增长；只有 Semaphore 而没有控制海量待处理对象的内存。

- 异常边界错误：不调用 Future.result；gather 默认异常传播理解错误；吞掉 CancelledError 或所有 Exception。
- 数据正确性错误：金额先转 float；把本轮 seen 当成持久化幂等；重试范围包含已成功保存步骤。

复习清单

- 能独立解释并手写 Order 的 len、for、in、索引和切片协议。
- 能说明 eq/hash 一致性、frozen 值对象、MRO 和 super 的真实执行顺序。
- 能根据 I/O/CPU 瓶颈选择 ThreadPoolExecutor、asyncio 或 ProcessPoolExecutor。
- 能写出 Future→业务 ID 映射、as_completed 和单任务异常隔离。
- 能解释 await 不等于并发，并使用 gather/TaskGroup 管理任务生命周期。
- 能正确使用 Semaphore、Queue(maxsize)、哨兵、task_done 与 join。
- 能为接口设置超时与有限重试，并明确哪些异常不重试。
- 能实现 Decimal 清洗、正整数 quantity、业务键去重、2/2/1 分批与幂等保存。
- 能为正常、超时、业务失败、并发上限和重复执行编写断言测试。

综合练习与参考答案

练习 1（基础）

题目：为 Cart 实现 len、for、in 和切片；in 按 SKU 忽略空格与大小写。

> **参考答案 / 解题思路**
> 参考思路：内部保存 list[Item]；__len__ 和 __iter__ 直接委托列表；__contains__ 先 isinstance 再标准化 SKU；__getitem__ 接收 int | slice，并用 overload 标明两种返回类型。

练习 2（线程池）

题目：并发请求 8 个同步站点，最多 4
个线程；任一站点失败不影响其他站点，并汇总成功数、失败站点和订单总数。

> **参考答案 / 解题思路**
> 参考思路：先把 8 个任务全部 submit；建立 future_to_site 字典；遍历 as_completed；在 future.result() 周围单独捕获预期接口异常；统计集中在主线程完成。

练习 3（asyncio）

题目：处理 30 个异步请求，接口并发不超过 5，每个请求超时 1 秒；分别统计成功、业务失败和超时。

> **参考答案 / 解题思路**
> 参考思路：每个请求函数内使用 async with semaphore；用 asyncio.timeout(1) 或 wait_for 限制接口阶段；任务内部将预期异常转换成状态，或 gather(return_exceptions=True) 后分类。

练习 4（数据与幂等）

题目：原始订单含 amount、quantity、SKU 和重复 order_id。实现清洗、校验、去重、每批 3 条保存，并证明重复执行不会新增。

参考答案 / 解题思路
参考思路：Decimal(str(value)).quantize(Decimal('0.01'), ROUND_HALF_UP)；quantity 拒绝 bool 且 >0；本轮 set 处理重复；数据库/仓储以 (site_id, order_id) 为唯一键；两次运行应 valid 可相同而第二次 saved=0。

练习 5（综合设计）

题目：设计 1000 店铺的分页订单同步，不一次性把所有订单放进内存，并支持断点续传。

参考答案 / 解题思路
参考思路：分页生产者按并发/限流拉取，把页面写入有界
Queue；固定消费者清洗并累积小批次保存；唯一索引/UPSERT
保证幂等；成功提交后持久化游标；失败记录可重放信息；增加重试抖动、指标、告警和优雅取消。

验收结果与学习文件状态

文件	验收结果
day_11_homework.py	编译通过；对象协议、OrderKey 去重、MRO、组合缓存、Money 舍入/比较断言通过
day_12_homework.py	编译通过；线程池成功/失败隔离、50,000 次加锁计数、4 个进程任务通过
day_13_homework.py	编译通过；顺序约 5.1s、并发约 1.0s；最大并发 4；8 成功/1 失败/1 超时；Queue 100 条恰好一次；to_thread 对比通过
day_14_homework.py	编译通过；2 次超时后成功；最大并发 3；批次 2/2/1；首次 saved=5、第二次 saved=0

最终结论
第 11-14 天对应文件齐全。讲义文件为注释型学习笔记，不能直接产生运行效果；四份作业已按保留题目的要求改为正确、可运行并带验收断言的版本。